

Introduction to computer science in **Python**

Fall 2024

**Department of Software and Information
Systems Engineering**

Ben-Gurion University of the Negev

Topic 11: Abstract Data Types (ADT)

Today: data structures

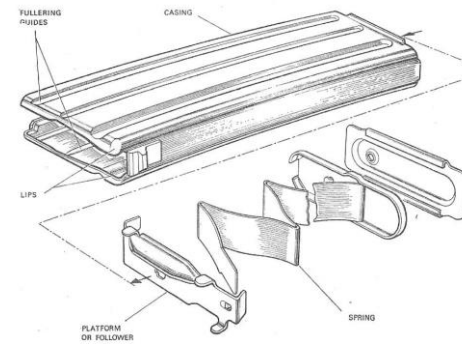
- A **data structure organizes** data to support specific operations.
- Operations are divided into two types:
 - **Queries**, like search, finding minimum
 - **Mutations**, like insertion, deletion, modification
- Built-in Python data structures: strings, tuples, lists, dictionaries
- The choice of data structures for a particular problem depends on desired operations and complexity constraints (time and memory)
- The term **Abstract Data Type (ADT)** emphasizes the point that the user (client) needs to know what **operations** may be used, but not **how** they are **implemented**

ADT examples

- Stack
- Queue
- Linked lists
- Binary Search Tree
- Graphs

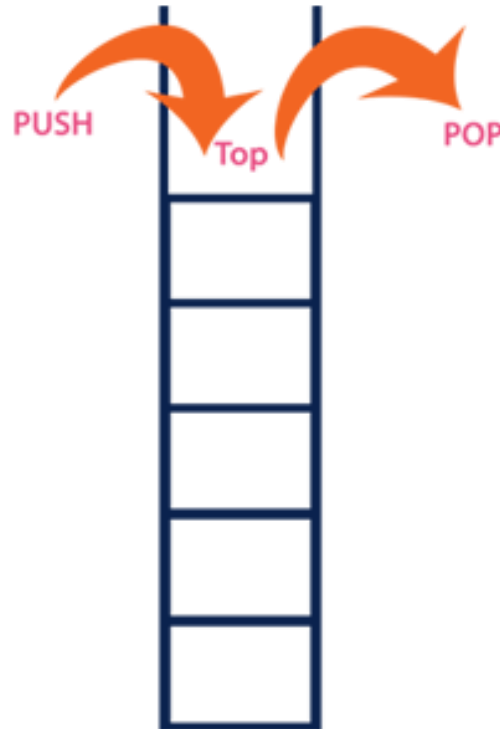
Stack

g() ->h()
f() ->g()
main -> f()



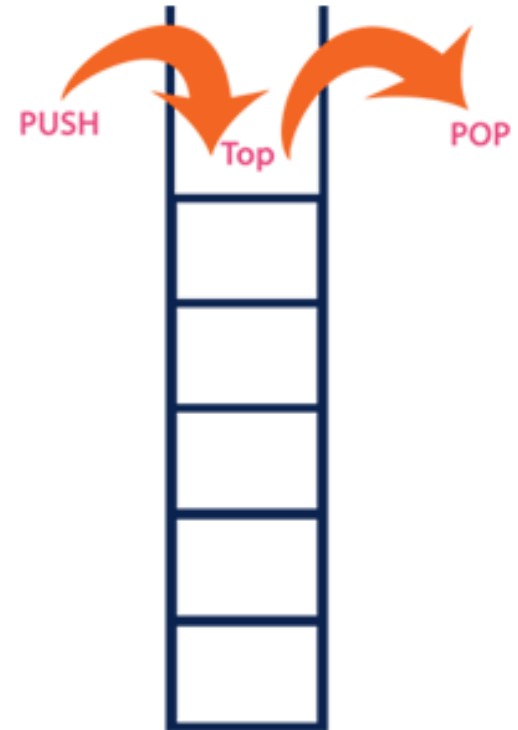
Stack

- Data structure in which the insertion and deletion operations are performed at one end
- a new element is added at top of the stack and an element is removed from the top of the stack
- **LIFO** (Last In First Out)
- Can be useful for:
 - String reversal
 - Undo operation
 - Parenthesis balance check
 - Functions stack



Stack operations

- `push(val)`
- `pop()`
- `peek()`
- `is_empty()`
- `size()`
- How would we implement it?



Stack implementation with a list

```
class MyStack:
    def __init__(self):
        self.__s = []

    def push(self, item):
        self.__s.append(item)

    def pop(self):
        return self.__s.pop()

    def is_empty(self):
        return len(self.__s) == 0

    def __repr__(self):
        return "(class MyStack, " + repr(self.__s) + ")"
```

Using a stack

```
s = MyStack()
```

```
print(s)
```

```
s.push(5)
```

```
s.push(7)
```

```
print(s)
```

```
s.pop()
```

```
print(s)
```

```
(class MyStack, [])
```

```
(class MyStack, [5, 7])
```

```
(class MyStack, [5])
```

Example (also in topic #7): balanced parentheses

2020-21, moed A, Q1

שאלה 1 (24 נקודות)

בשאלה זו תממשו קוד שבודק האם ביטוי שמכיל סוגריים בלבד מוגדר כביטוי מאוזן.

סעיף א' (14 נקודות)

ביטוי מיוצג באמצעות מחרוזת המורכבת מתווים של סוגריים פותחים וסוגרים מהסוגים הבאים: {}, [], ו-(). ביטוי מאוזן הוא ביטוי המקיים את שני התנאים הבאים:

- לכל סמל פותח יש סמל סוגר תואם. כלומר, לא יהיה סמל פותח ללא סמל סוגר תואם ולהפך.
- זוגות הסוגריים התואמים מקוננים כהלכה. כלומר, עבור כל סוגר פותח יימצא סוגר תואם במקום המתאים.

נדגיש כי זוג סוגריים פותח וסוגר חייבים להיות מאותו הסוג. למשל, אם יש סמל פותח מסוג '(' הסוגר המתאים חייב להיות ')', סוגר מהסוג '[' או מהסוג '}' אינם מתאימים.

'(OOOO)' – balanced

'(((('))' – balanced

'({[]})' – balanced

'(((((((' - not balanced

'())' - not balanced

'{}'- not balanced

'[]'- not balanced

'{()}' - not balanced

ממשו את הפונקציה `balance(expression)`, אשר מקבלת כקלט ביטוי המכיל סוגריים מהסוגים שצוינו מעלה ומחזירה `True` אם הביטוי מאוזן ואחרת `False`. בסעיף זה מומלץ להשתמש באחד ממבני הנתונים הבאים על מנת לפתור את השאלה – מילון, מחסנית, תור או רשימה מקושרת. הנספח מכיל עבור כל מבנה נתונים את הממשק המתאים - רשימת השיטות שמבנה נתונים תומך בהם. **אינכם** נדרשים לממש את מבני הנתונים אלא רק תשתמשו בהם.

דוגמת ריצה:

```
>>> expression = '{()}'
```

```
>>> balance(expression)
```

```
True
```

```
>>> expression = '[( {} {} )]'
```

```
>>> balance(expression)
```

```
False
```

Solution with a stack

- Open parenthesis – push to the stack
- Close parenthesis – pop from the stack and check that the parentheses match

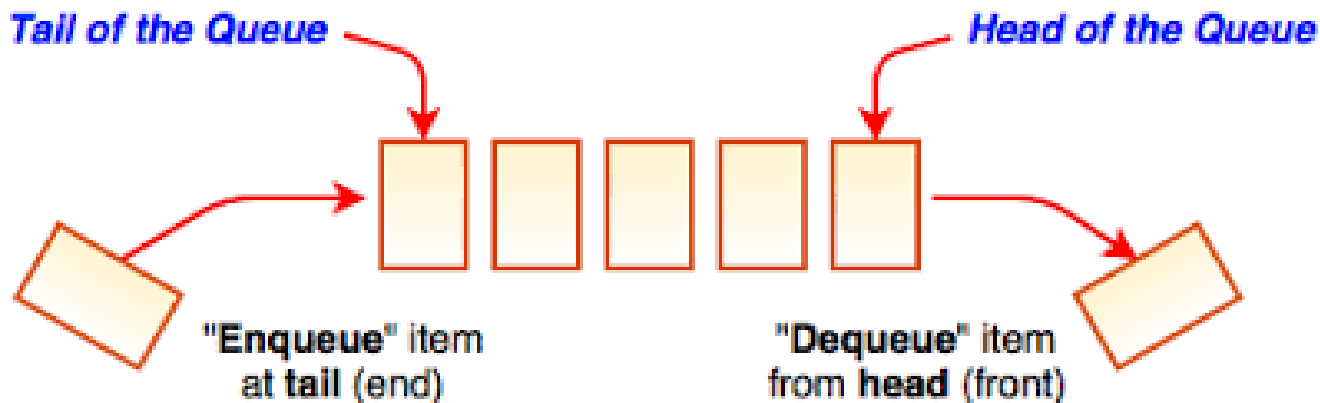
Implementation

```
def balance(expression):
    stack = MyStack()

    for char in expression:
        if char in ['(', '{', '[']:
            stack.push(char)
        else:
            if stack.is_empty():
                return False
            current_char = stack.pop()
            if current_char == '(':
                if char != ")":
                    return False
            if current_char == '{':
                if char != "}":
                    return False
            if current_char == '[':
                if char != "]":
                    return False
    if not stack.is_empty():
        return False
    return True
```

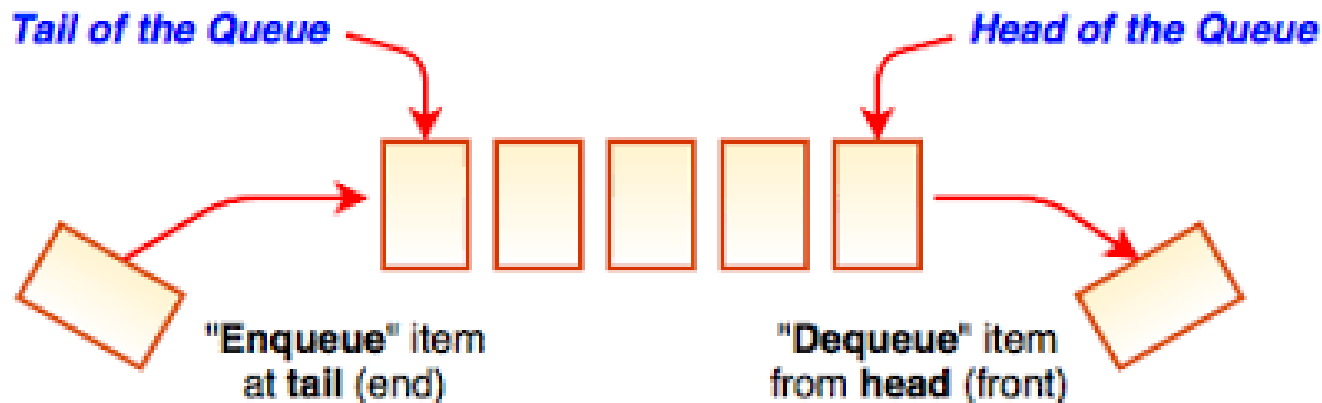
Queue

- One end used to insert data (*enqueue*), the other end to remove data (*dequeue*)
- **FIFO** (First In First Out) - the data stored first will be accessed first
- Can be useful for:
 - Path finding algorithms (solving mazes)
 - Resources allocation
 - Actual queues (bank, delivery, doctor appointment)



Queue operations

- `enqueue(val)`
- `dequeue()`
- `is_empty()`
- `size()`
- How would we implement it?



Queue implementation with a list

```
class MyQueue:
    def __init__(self):
        self.__items = []

    def enqueue(self, item):
        self.__items.insert(0,item)

    def dequeue(self):
        return self.__items.pop()

    def is_empty(self):
        return len(self.__items) == 0

    def size(self):
        return len(self.__items)

    def __repr__(self):
        return "(class MyQueue, " + repr(self.__items) + ")"
```

Using a queue

```
q = MyQueue()
for i in range(11):
    if i % 5 == 0:
        print(i,q)
    q.enqueue(i**2)
    if i % 5 != 0:
        print(i+1,q)
    print()
```

0 (class MyQueue, [])

1 (class MyQueue, [0])

5 (class MyQueue, [16, 9, 4, 1, 0])

6 (class MyQueue, [25, 16, 9, 4, 1, 0])

10 (class MyQueue, [81, 64, 49, 36, 25, 16, 9, 4, 1, 0])

11 (class MyQueue, [100, 81, 64, 49, 36, 25, 16, 9, 4, 1, 0])

```
print(q.dequeue(),q.dequeue(),q.dequeue(),q.dequeue())
```

0 1 4 9

Iterating over a queue with a generator

```
def queue_gen(queue):  
    while not queue.is_empty():  
        yield queue.dequeue()  
  
queue = MyQueue()  
for i in range(6):  
    queue.enqueue(i**2)  
  
for elem in queue_gen(queue):  
    print(elem)
```

0

1

4

9

16

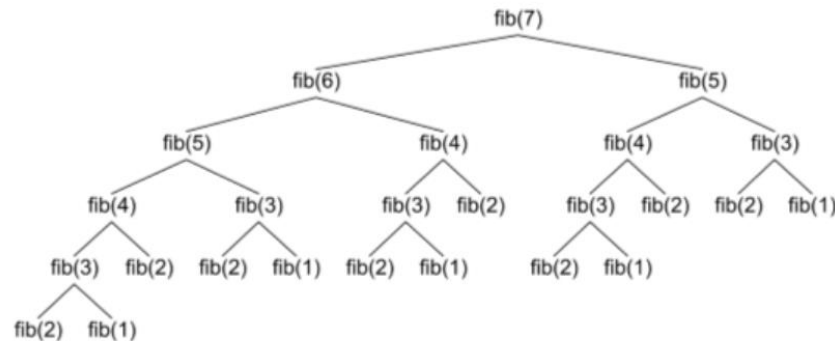
25

Abstract Data Types (ADT)

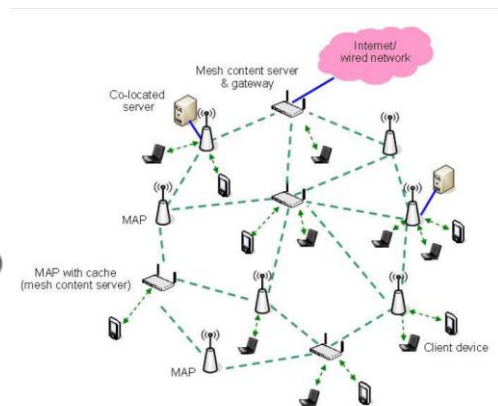
- “class of objects whose logical behavior is defined by a set of values and a set of operations” (Dalle & Walker, 1996)
- There are many ways to implement the same ADT
- Stack and Queue are examples for ADTs



<https://bizmeetstech.net/amazon-connect-queue-basics/>



<https://stackoverflow.com/questions/-no-noitanalpxe/35959100/noisrucer-iccanobif>

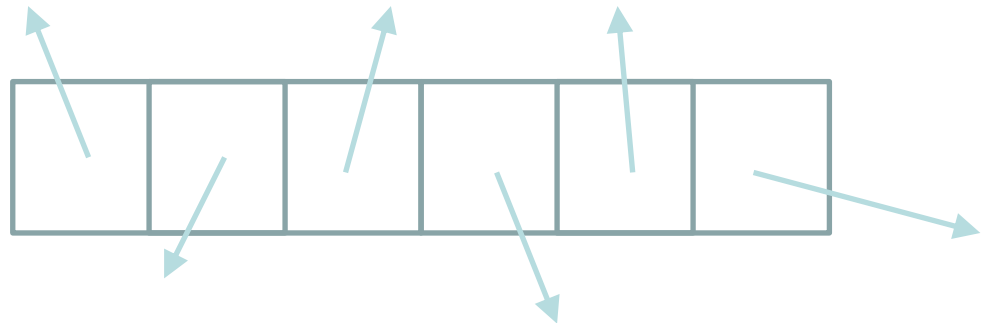


https://www.researchgate.net/figure/illustrates-an-infrastructure-WMNs-It-consists-of-mesh-access-points-MAPs-routers-and_fig221170162_1

- Interfaces

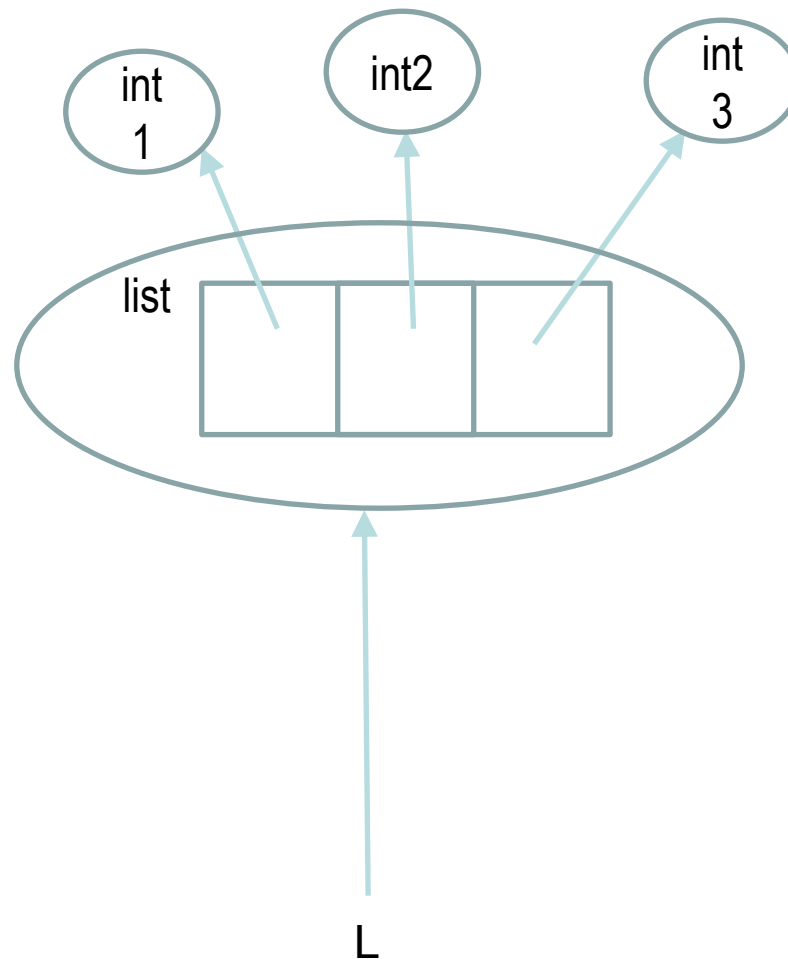
Implementation of lists in Python

- Python implements a **continuous** array of **pointers**
- Python keeps the address of this array in memory, and its length in a list head structure
- Accessing/modifying $a[i]$ is $O(1)$. If the address in memory of $lst[0]$ is a , then the address in memory of $lst[i]$ is $a+i$
- Pointers enable Python's lists to contain objects of heterogeneous types



Implementation of lists in Python

```
>>> L = [1,2,3]
```



Implementation of lists in Python

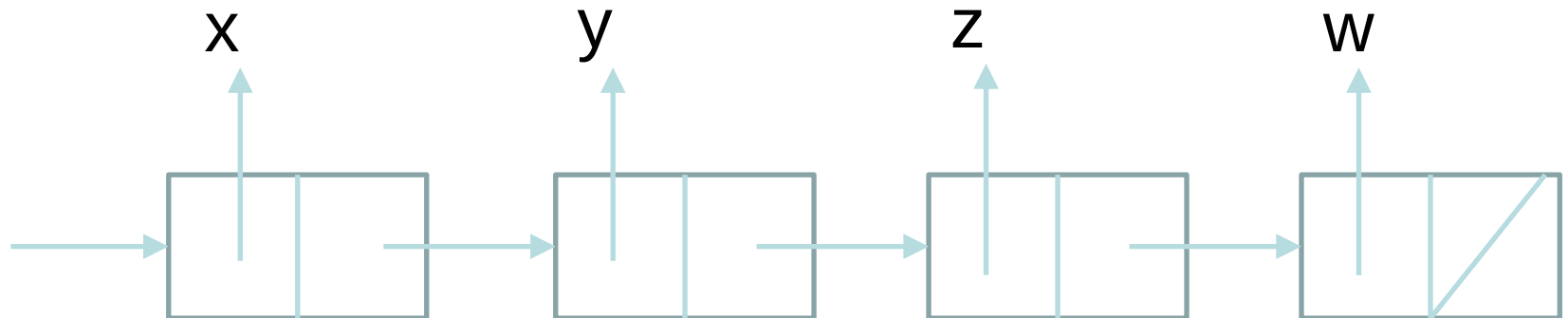
- Contiguous storage of addresses must be maintained when a list **evolves**
- Inserting an item at location $i \rightarrow O(n)$ operations
- Using up all of the memory block allocated for the list $\rightarrow$ allocating a block of **larger size** (possibly new location)
- **Improving the performance**: extra space is allocated right away, so the next few times do not require an actual resizing
- Official source: [How are lists implemented?](#)

Linked structures

We will implement a new data structure, called **Linked List**, and compare it to Python's built-in list structure. Later on we will discuss another linked structure, **Binary Search Trees**

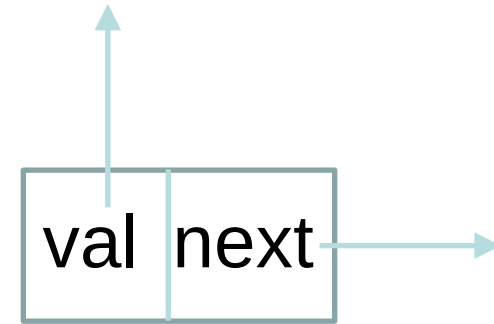
Linked List

- Instead of using a contiguous block of memory, specify for each item, the memory location of the **next** item in the list
- Insertion at $O(1)$
- We can represent this graphically:



Linked list implementation: class Node

- Two classes. One for nodes in the list, and another one to represent a list
- Class Node is very simple, holding just two fields



```
class Node:
    def __init__(self, val):
        self.value = val
        self.next = None

    def __repr__(self):
        return "[" + str(self.value) + "," + str(id(self.next)) + "]"
#This shows pointers as well for educational purposes
```

Using class Node

```
n1 = Node(10)
n2 = Node(20)
n1
```

```
[10, nxt:140706590664832]
```

```
n1.next = n2
n1
```

```
[10, nxt:1674932143152]
```

```
n1.next.value
```

```
20
```

```
n2.next.value
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-12-70c31197ea62> in <module>
----> 1 n2.next.value
```

```
AttributeError: 'NoneType' object has no attribute 'value'
```

class LinkedList

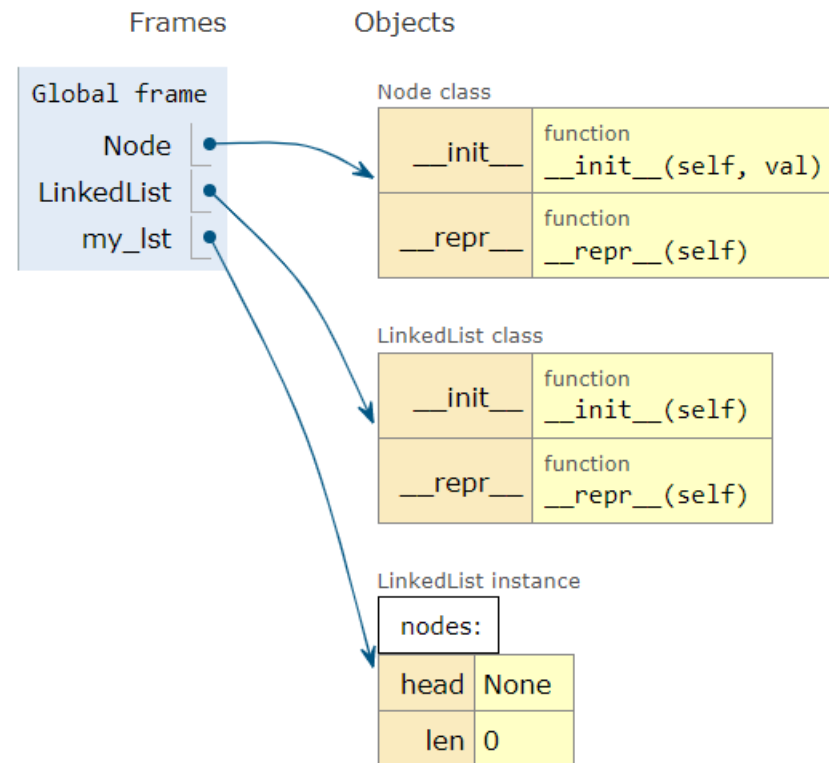
- Holds the head of the list (of type Node) and the list's length

```
class LinkedList:
    def __init__(self):
        self.head = None
        self.len = 0

    def __repr__(self):
        out = " nodes:"
        p = self.head
        while p != None:
            out += str(p) + " "
            p = p.next
        return out
```

Memory view

```
my_lst = LinkedList()
```



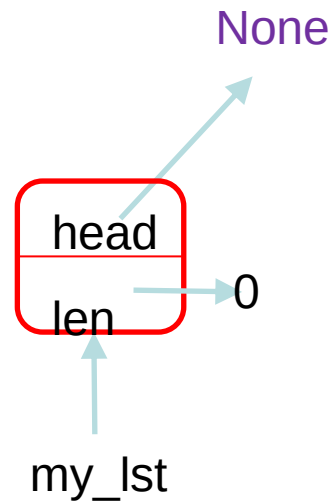
Insertion at the start of the linked list

```
def add_at_start(self, val):  
    ''' add node with value val at the list head '''  
    new_node = Node(val)  
    if self.len == 0:  
        self.head = new_node  
    else:  
        tmp = self.head  
        self.head = new_node  
        self.head.next = tmp  
    self.len += 1
```

Note: time complexity is $O(1)$ in the worst case!

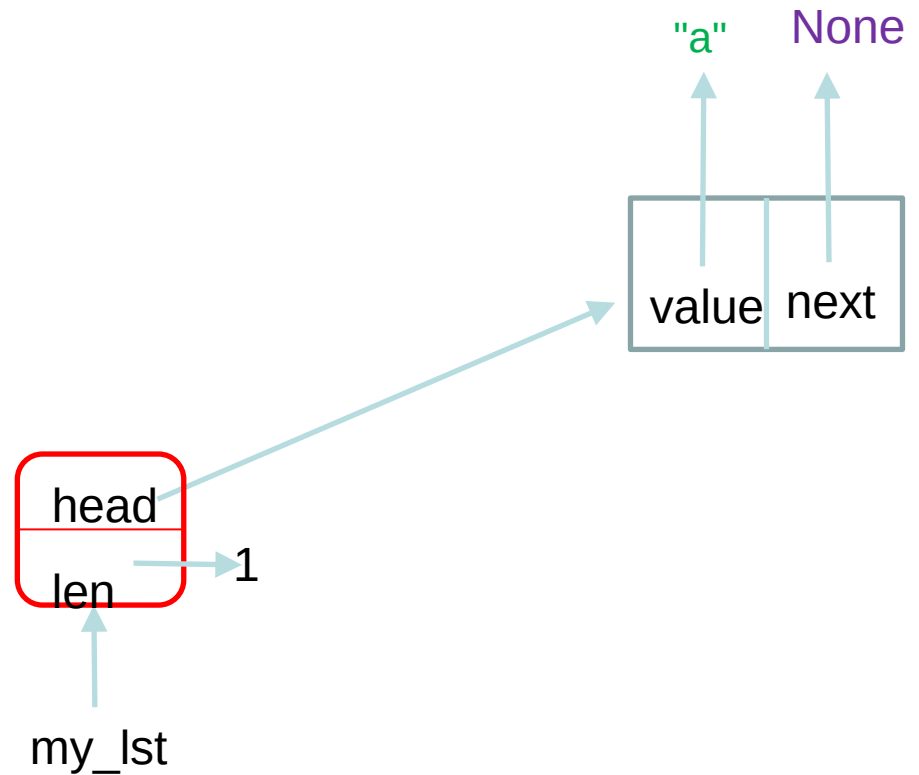
Memory view

```
>>> my_lst = Linked_list()
```



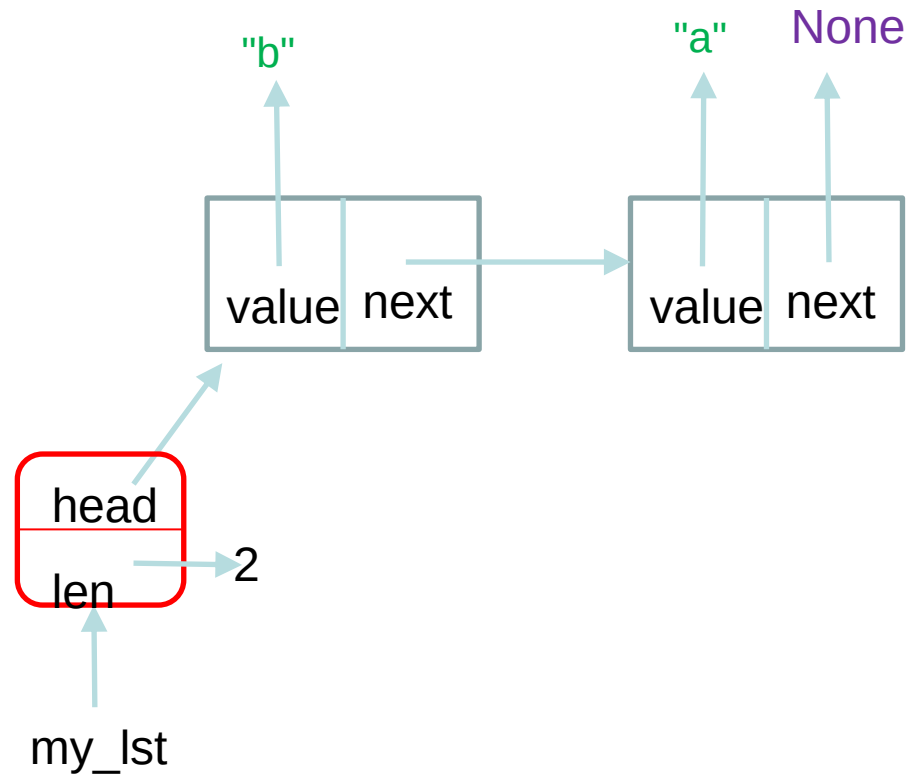
Memory view

```
>>> my_lst.add_at_start("a")
```



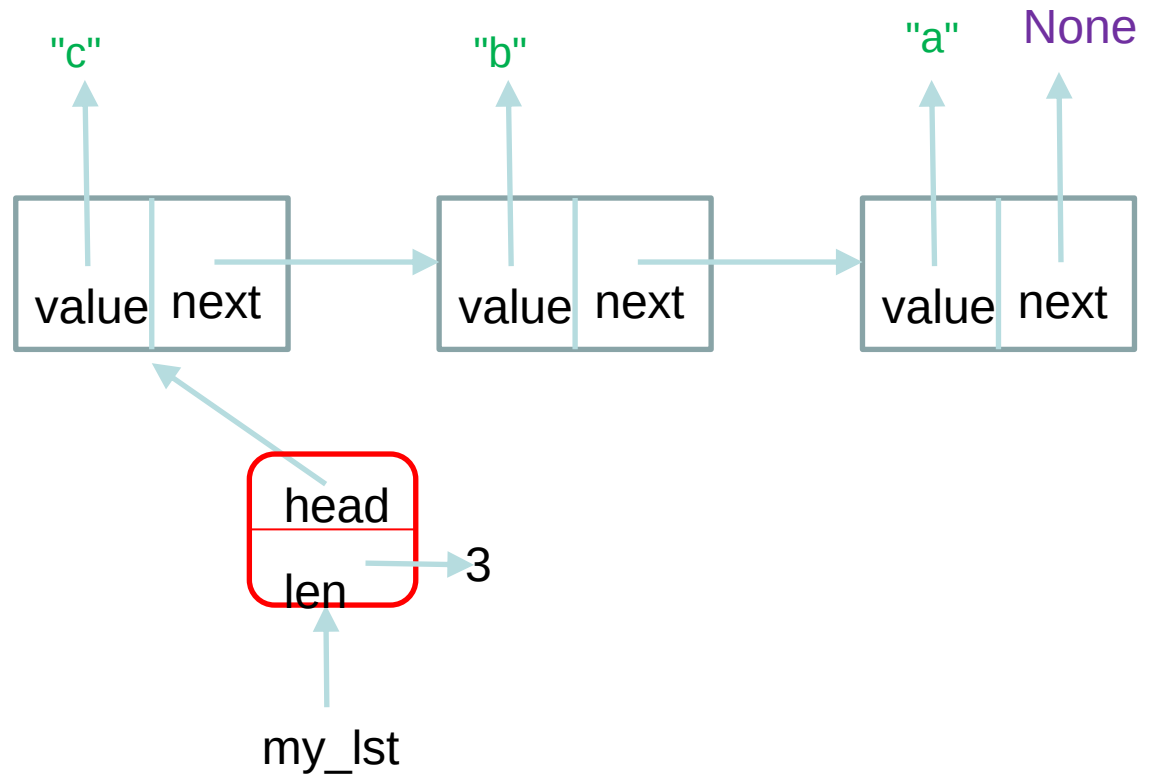
Memory view

```
>>> my_lst.add_at_start("b")
```



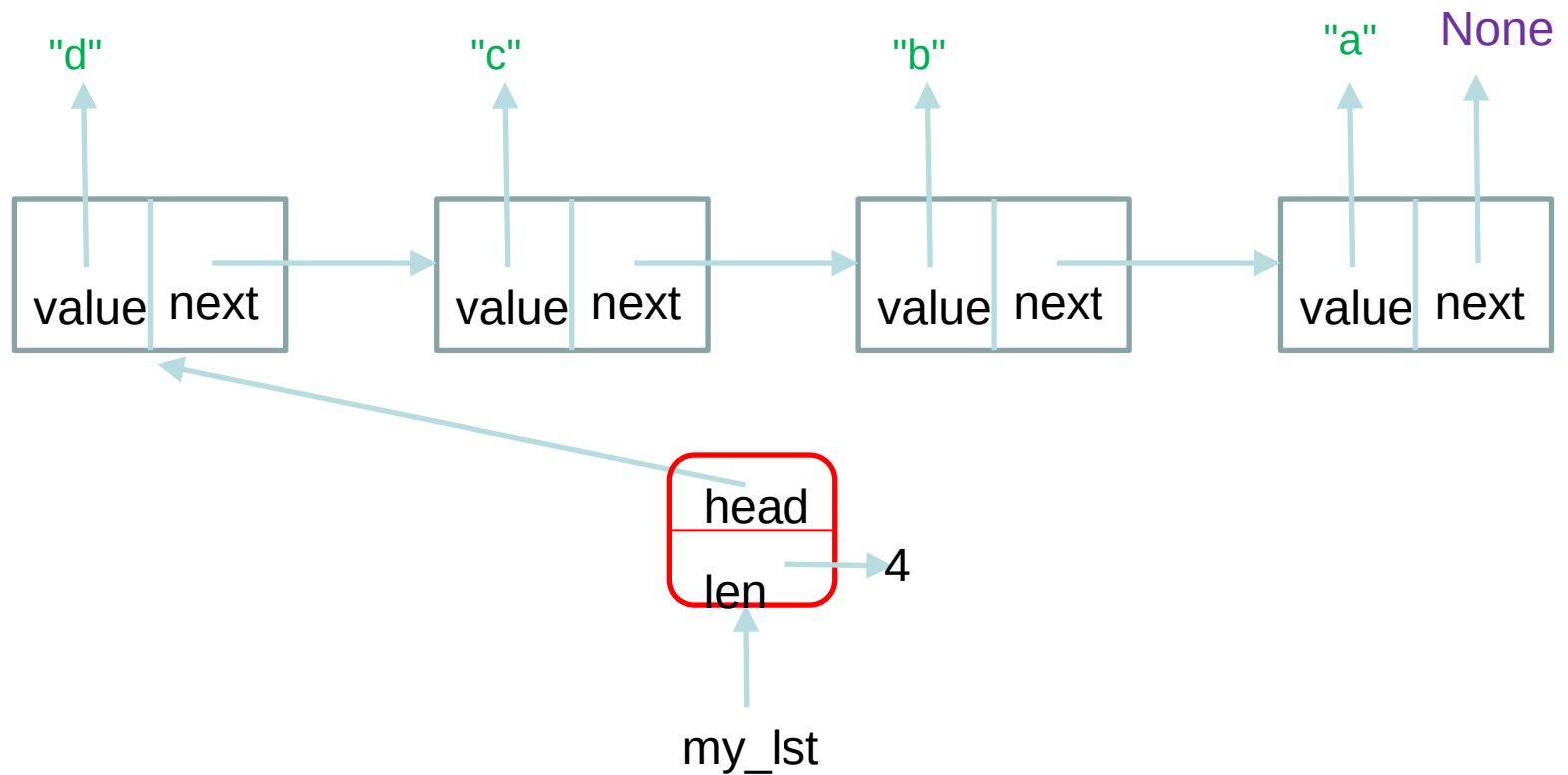
Memory view

```
>>> my_lst.add_at_start("c")
```



Memory view

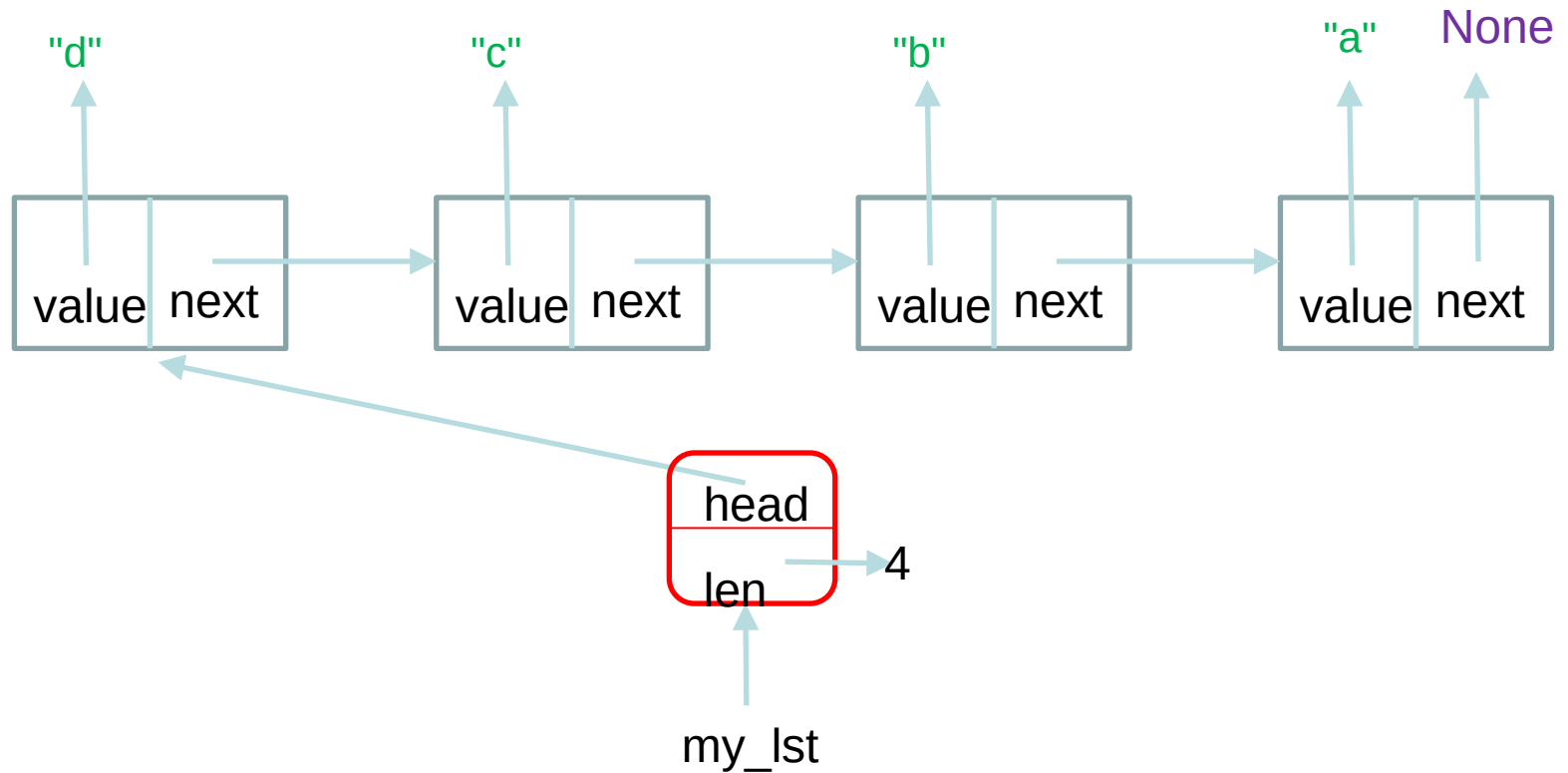
```
>>> my_lst.add_at_start("d")
```



Memory view

```
>>> print(my_lst)    #calls __repr__ of class Linked_list  
[d,44602768] [c,44602736] [b,44602096] [a,1749231768]
```

```
>>> id(None)  
1749231768
```



Length of the linked list

```
def __len__(self):  
    ''' called when using Python's len() '''  
    return self.len
```

```
>>> len(my_lst)
```

```
4
```

```
>>> my_lst.__len__() #same
```

```
4
```

- The time complexity is $O(1)$
- The field len must be updated when inserting / deleting!

Linked list indexing

```
def __getitem__(self, loc):  
    ''' called when using L[i] for reading  
        return node at location 0<=loc<len '''  
    p = self.head  
    for i in range(0, loc):  
        p = p.next  
    return p
```

```
>>> my_lst[1].value  
'c'
```

```
>>> my_lst.__getitem__(1).value #same  
'c'
```

- The argument loc must be between 0 and the length of the list
- Time complexity: $O(\text{loc})$. In the worst case $\text{loc} = n$

Linked list indexing

```
def __setitem__(self, loc, val):  
    ''' called when using L[loc]=val for writing  
        assigns val to node at location 0<=loc<len '''  
    p = self.head  
    for i in range(0, loc):  
        p = p.next  
    p.value = val  
    return None
```

```
>>> my_lst[1] = 999 #same:my_lst.__setitem__(1,999)  
>>> print(my_lst)  
[d,44602768] [999,44602736] [b,44602096] [a,1749231768]
```

- The argument loc must be between 0 and the length of the list
- Time complexity: $O(\text{loc})$. In the worst case $\text{loc} = n$

Insertion at a given location of a linked list

```
def insert(self, loc, val):  
    ''' add node with value val at location 0<=loc<len of the list '''  
    new_node = Node(val)  
    if loc == 0:  
        new_node.next = self.head  
        self.head = new_node  
    else:  
        p = self.head  
        for i in range(1, loc):  
            p = p.next  
        tmp = p.next  
        p.next = new_node  
        p.next.next = tmp  
    self.len += 1
```

- Time complexity: $O(\text{loc})$. Worst case $\text{loc} = n$.
- When loc is 0, we get the same effect as add_at_start:

Find a value in the linked list

```
def find(self, val):  
    ''' find (first) node with value val in list '''  
    p = self.head  
    # loc = 0      # in case we want to return the location  
    while p != None:  
        if p.value == val:  
            return p  
        else:  
            p = p.next  
            #loc=loc+1    # in case we want to return the location  
    return None
```

- Time complexity: worst case $O(n)$, best case $O(1)$

Delete element from the linked list

```
def delete(self, loc):  
    ''' delete element at location  $0 \leq \text{loc} < \text{len}$  '''  
    if loc == 0:  
        self.head = self.head.next  
    else:  
        p = self.head  
        for i in range(1, loc):  
            p = p.next  
            if p.next != None: #p cannot be None  
                p.next = p.next.next  
        self.len -= 1
```

- Time complexity: $O(\text{loc})$. In the worst case $\text{loc} = n$.
- Python Garbage collector will “remove” the deleted item (assuming there is no other reference to it) from memory
- Note that in some languages (e.g., C, C++) the programmer is responsible to free memory

Delete element from the linked list

- How would you delete an item with a given **value** (not location)?
- (Small) **technical inconvenience**: in order to delete an item, we need to access the item **before** it
- A possible solution: two points (prev, cur)
- Another possible solution: keep a **2-directional linked list** (each node points both to the **next** and the **previous** node)
- This requires $O(n)$ additional memory

An extended `__init__`

- Initialization of a LinkedList object from an existing Python's **sequence** (e.g. list, string, tuple)

```
def __init__(self, seq=None):
    self.head = None
    self.len = 0
    if seq != None:
        for x in seq[::-1]:
            self.add_at_start(x)
```

- We employ `add_at_start(ch)` for efficiency reasons

```
>>> L = LinkedList("abc")
```

```
>>> print(my_lst)
```

```
[a, 42430064] [b, 42430032] [c, 1749231768]
```

Linked Lists vs. Python Lists:

Complexity of Operations

- When we have a pointer to an element, **inserting** an element **after it** requires just $O(1)$ operations. Compare to $O(n)$ for python lists
- **Deletion** of a **given** item requires $O(1)$ time, assuming we have access to the previous item. Compare to $O(n)$ for Python lists
- **Accessing** the **i-th** item requires $O(i)$ time. Compare to $O(1)$ for Python lists

Sorted, Linked Lists

- Consider **sorted linked lists**. What would be improved this way? What would not?
- Maintain a sorted linked list by always inserting an item in its correct location.

```
def insert_ordered(self, val):  
    ''' assume self is an ordered list,  
        insert Node with val in order '''  
    new_node = Node(val)  
    p = self.head  
    if p == None or val <= p.value:  
        new_node.next = p  
        self.head = new_node  
    else:  
        q = p.next  
        while q != None and q.value < val:  
            p = q  
            q = q.next  
  
        p.next = new_node  
        new_node.next = q  
    self.len += 1
```

Searching in an sorted linked list

- We cannot use binary search in a sorted linked list because random access to the i'th element is not possible in constant time

```
def find_ordered(self, val):  
    ''' assume self is an ordered list,  
        find Node with value val '''  
    p = self.head  
    while p != None and p.value < val:  
        p = p.next  
    if p != None and p.value == val:  
        return p  
    else:  
        return None
```

Looping through linked lists via index access

```
for i in range(len(lst)):
    lst[i]
```

- Time complexity $O(n^2)$

Iterating through linked lists

- Iterable LinkedList
- Implement `__iter__` and `__next__`
- In LinkedList:

```
def __iter__(self):  
    self.runner = self.head  
    return self  
  
def __next__(self):  
    if self.runner is None:  
        raise StopIteration  
    res = self.runner  
    self.runner = self.runner.next  
    return res
```

Iterating through linked lists

```
for i in lst:  
    i
```

- Time complexity $O(n)$

Traversing through linked lists with a generator

```
def generator_linked_list(lst):  
    ptr = lst.head  
    while ptr:  
        yield ptr  
        ptr = ptr.next  
  
for i in generator_linked_list(lst):  
    i
```

LinkedList index access vs. iteration

```
from timeit import default_timer as timer
lst = LinkedList()
for i in range(4000):
    lst.add_at_start(i)

start1 = timer()
for i in range(len(lst)):
    lst[i]
end1 = timer()
print('index:', (end1 - start1))

start2 = timer()
for i in lst:
    i
end2 = timer()
print('Iteration:', (end2 - start2))
```

index: 0.40004649999999167

Iteration: 0.00125589999997054343

Implementing a stack using a linked list

```
class LinkedListStack:
    def __init__(self):
        self.__items = LinkedList()

    def push(self, item):
        self.__items.add_at_start(item)

    def pop(self):
        res = self.__items[0]
        self.__items.delete(0)
        return res

    def peek(self):
        return self.__items[0]

    def is_empty(self):
        return len(self.__items) == 0

    def size(self):
        return len(self.__items)

    def __repr__(self):
        return "(class LinkedListStack, " + repr(self.__items) + ")"
```

Implementing a stack using a linked list

```
s = LinkedListStack()  
t = LinkedListStack()
```

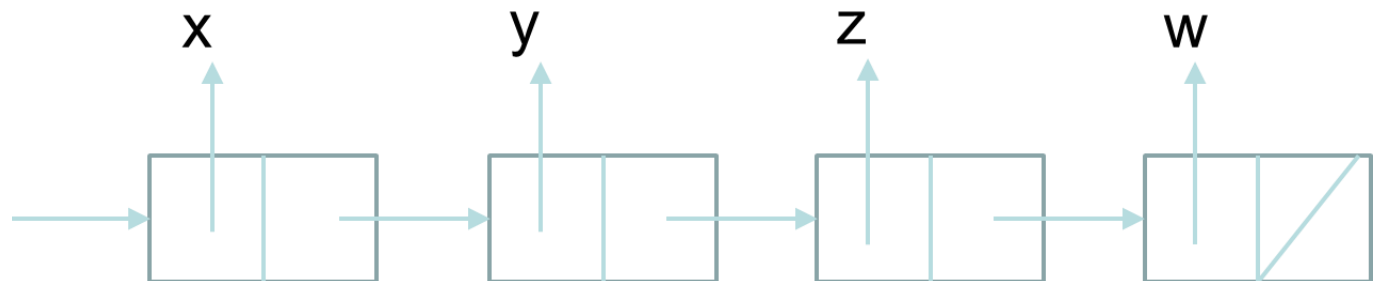
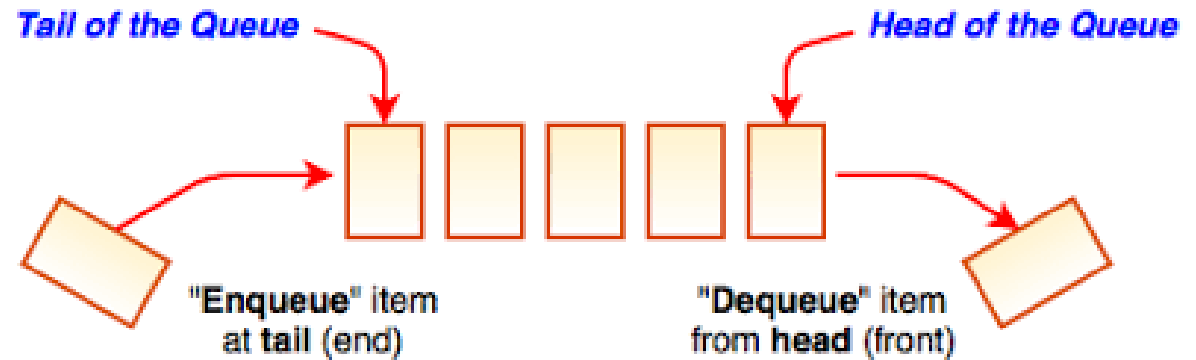
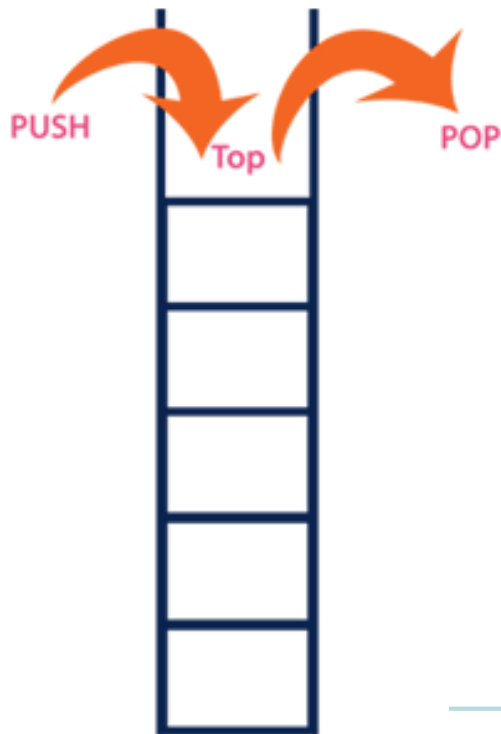
```
print(s)  
s.push(5)  
s.push(7)  
print(s)  
s.pop()  
print(s)
```

```
(class LinkedListStack, nodes: )  
(class LinkedListStack, nodes: [7, nxt:1674967655520] [5, nxt:140706590664832] )  
(class LinkedListStack, nodes: [5, nxt:140706590664832] )
```

Linked data structures

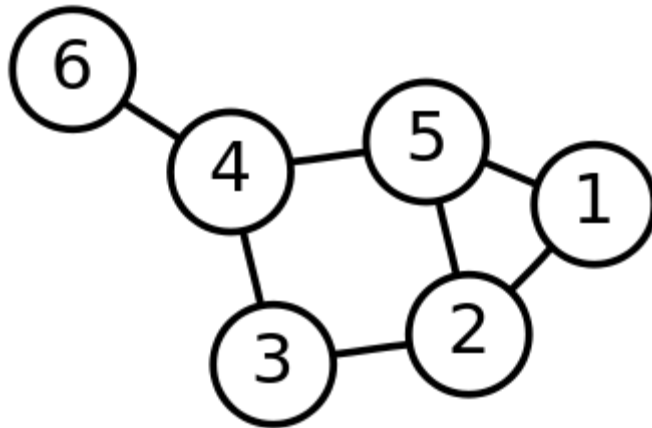
- Linked lists are just the simplest form of linked data structures
- **Doubly-linked** lists: nodes include a pointer to the preceding and to the next item
- **Binary trees**: node points to its left and right child
- **Graphs**: node can point to multiple other nodes
- we'll see these next...

ADTs: stack, queue, linked list



Graphs

- A graph is a structure with **Nodes** (or vertices) and **Edges**. An edge connects two nodes
- In **directed graphs**, edges have a direction
- In undirected graphs, the edges have no direction



Example of undirected graph

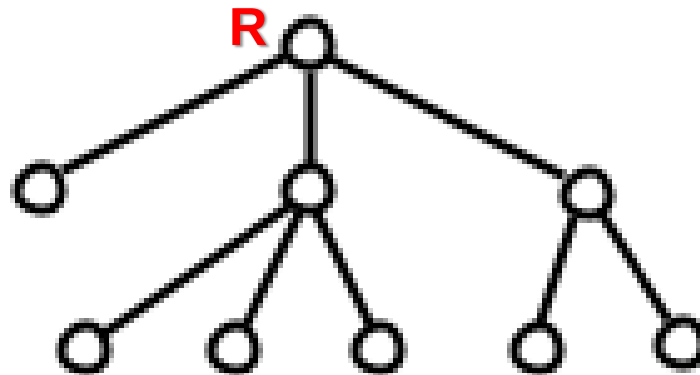
Source: wikipedia

Trees

- **Tree:** “undirected graph in which any two vertices are connected by exactly one path. Every acyclic connected graph is a tree, and vice versa.” (Wikipedia)
- We will only discuss **rooted trees**

Rooted Trees

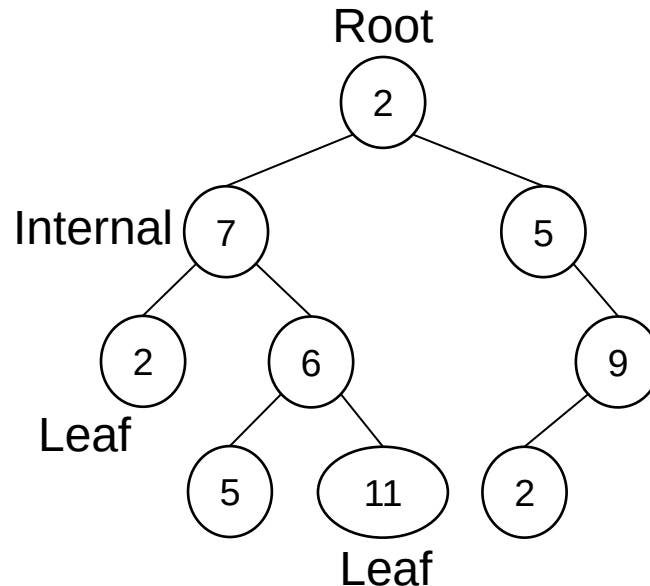
- A *rooted tree* is a tree in which a specific node is designated as a special node call the *root* of the tree
- Nodes connected by edges to the root are considered its child nodes
- The root is the parent node of theses same nodes
- Note: these definitions are recursive



Adapted from Wikimedia Commons

Rooted trees – basic notions

- A **directed edge**: an edge from a parent to a child
- The root node of a tree is the (unique) node with no parents
- A leaf node has no children
- Non leaf nodes are called internal nodes

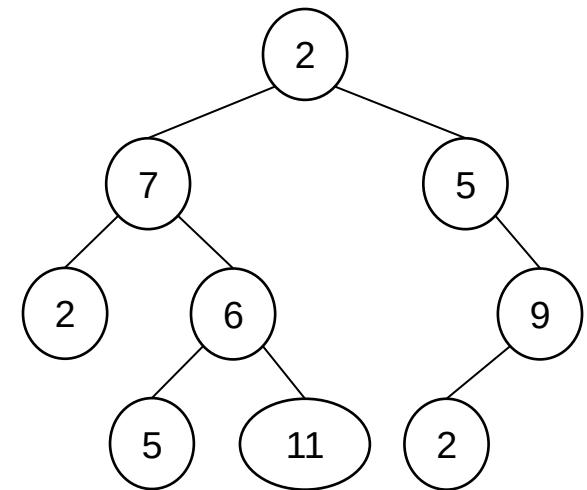


Rooted trees – basic notions

- A node p is an *ancestor* of a node q if p exists on the (only) simple path from the root node to node q
- A node q is a *descendent* of a node p if p is an *ancestor* of q
- The *out-degree* of a node is the number of edges leaving that node
- All the leaf nodes have an out-degree of 0

Rooted trees – basic notions

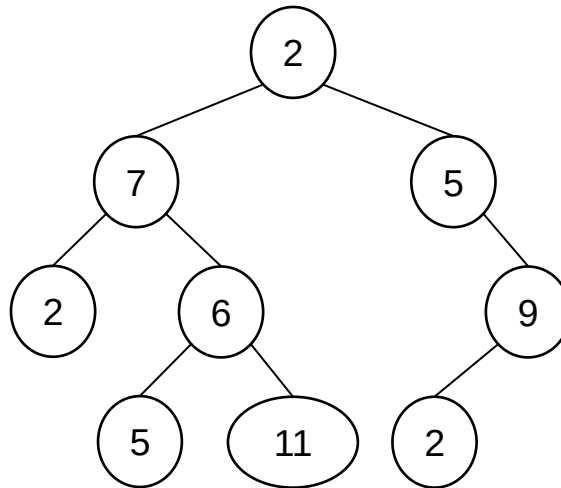
- The *depth** of node p is the length of the simple path in the tree from the root to p
- The *height** of node p is the length of the longest simple path from the p to a descendent leaf node in the tree
- The *height* of a tree is the height of its root
 - For a tree with only one node (the root)
 $height = 0$
 - For an empty tree with no nodes
 $height = -1$



* Depth/height can be defined as length in nodes, i.e. +1 to the current values

Example

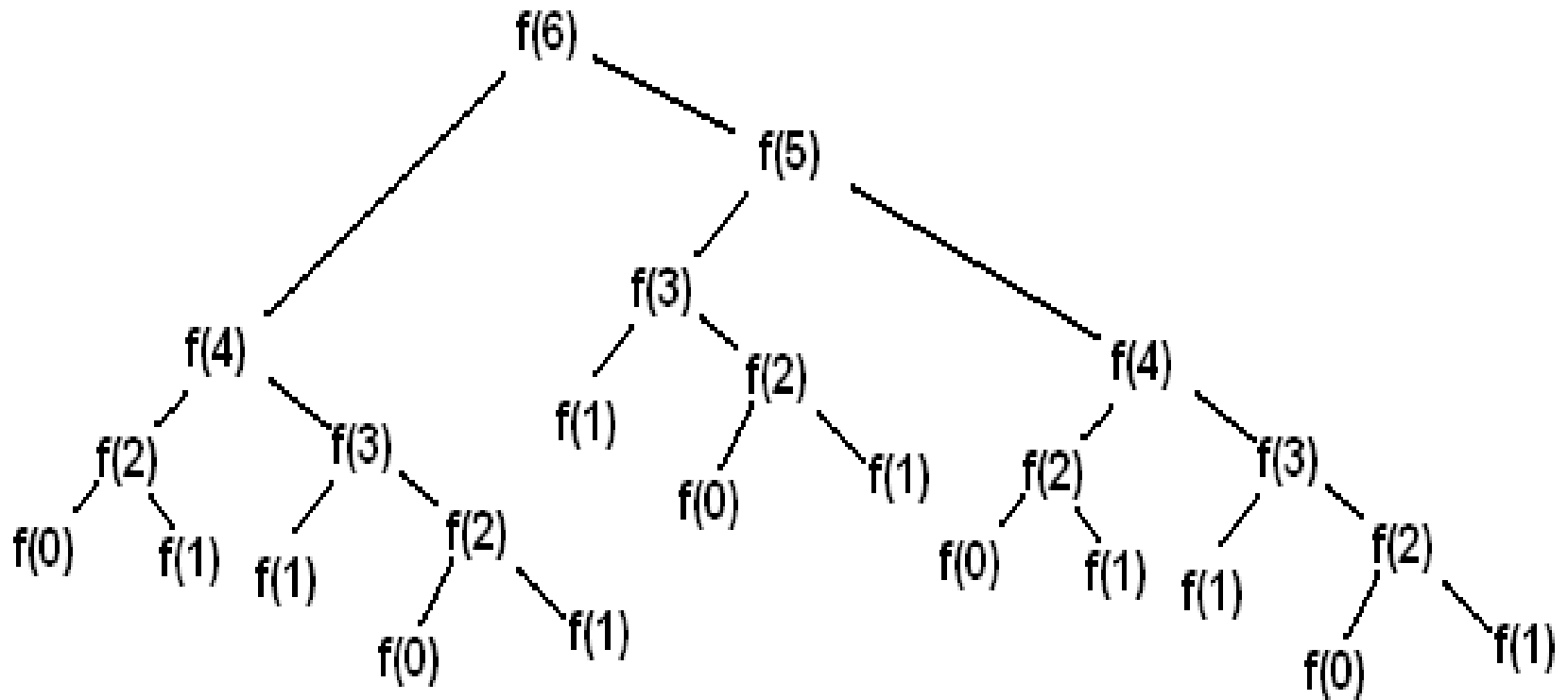
- Root is labeled 2
- Height is 3
- Node 11 is a descendent of 7, but not of (either of the two nodes labeled) 5



Applications of trees

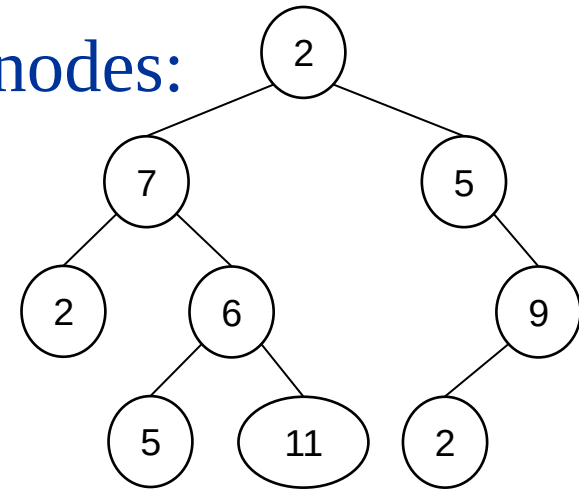
- Trees may be used to represent:
 - Arithmetic expressions
 - Games
 - Evolution
 - Recursion
 - Visualizing algorithms
 - Compilers

Example: call tree for Fibonacci $f(n)$



Rooted binary trees

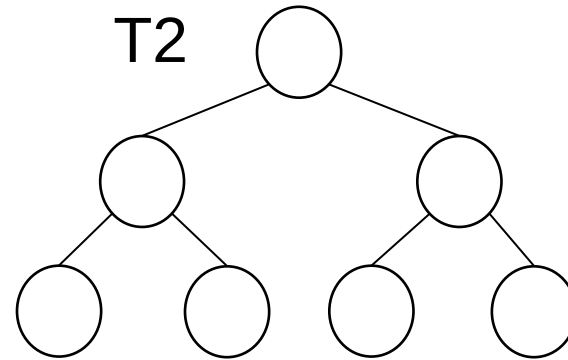
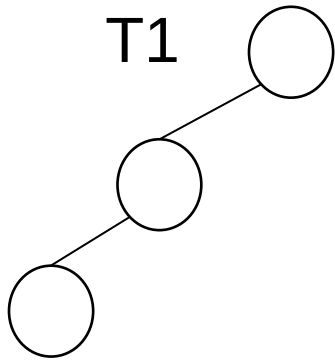
- Definition: a rooted binary tree
 - contains no nodes (empty tree), or
 - is comprised of three **disjoint** sets of nodes:
 - a **root** node,
 - a binary tree called its **left subtree**
 - a binary tree called its **right subtree**



- Note:
 - This is a **recursive definition**
 - This is a special case of rooted trees, in which each node has **at most 2 children**

Properties of binary trees

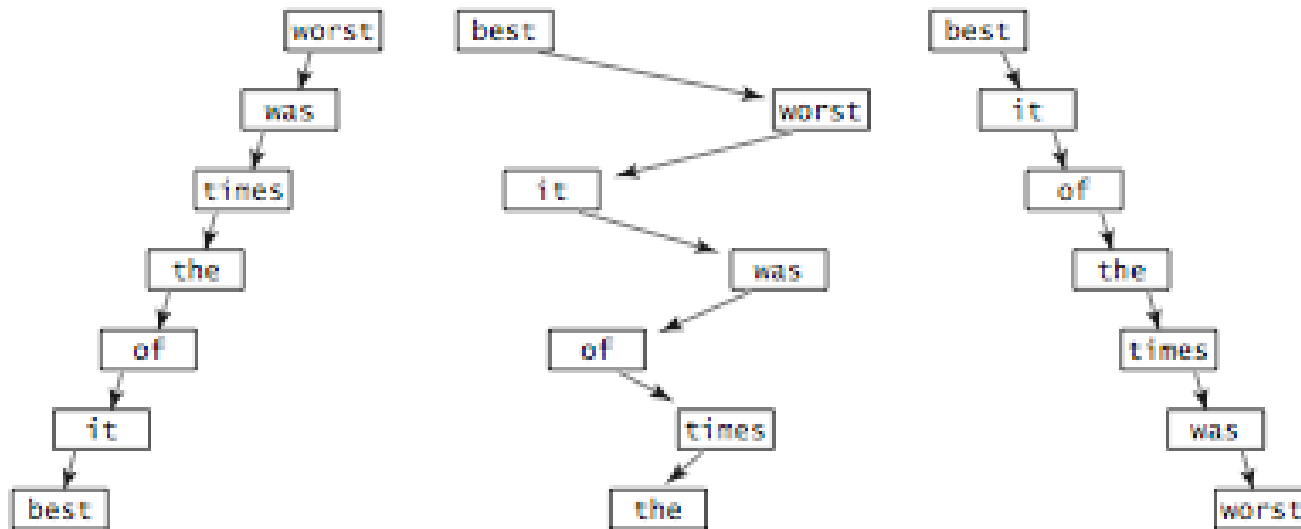
- The number of nodes in a binary tree of height h satisfies $h + 1 \leq n \leq 2^{h+1} - 1$
- A binary tree of height 2 has between $2 + 1 = 3$ and $2^3 - 1 = 7$



- More examples in slides to follow

Totally unbalanced binary tree

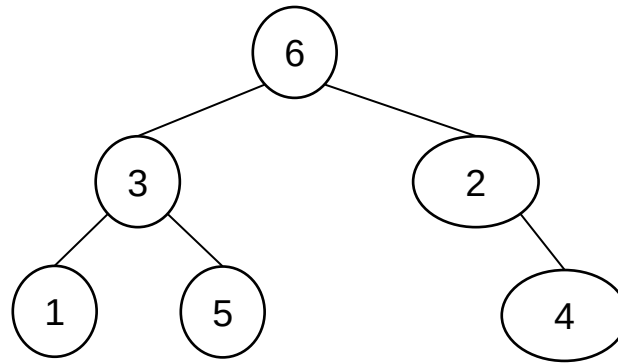
Each node has only **one non empty subtree**. The depth of such a totally unbalanced tree with n nodes is $d = n - 1$



Worst case (totally unbalanced) BSTs

Binary tree example

Shown are the **keys**, each node also has a **value** (not shown)



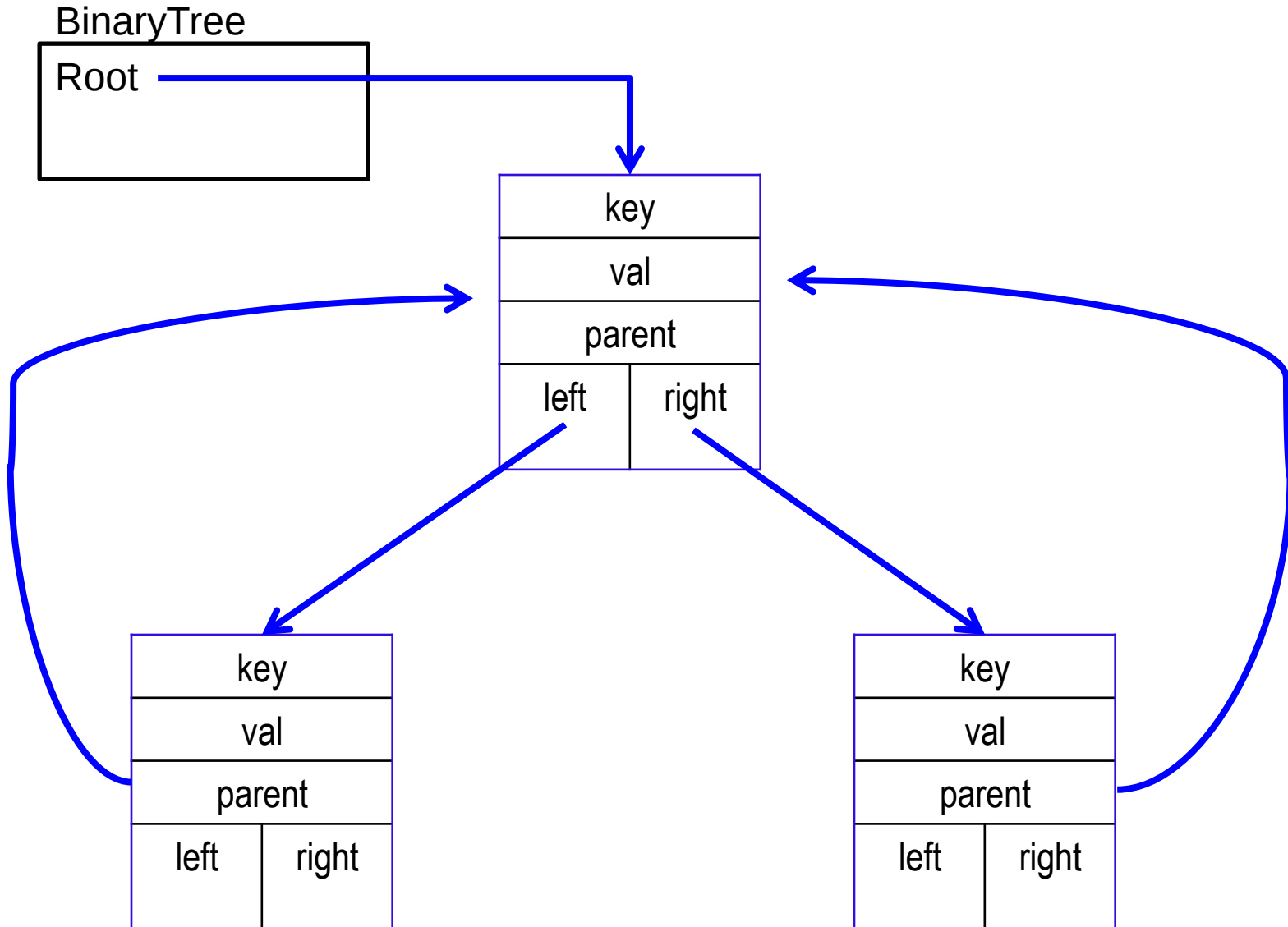
Binary tree implementation: tree node

```
class TreeNode():
    def __init__(self, key, val):
        self.key = key
        self.val = val
        self.left = None
        self.right = None
        self.parent = None

    def __repr__(self):
        return "(" + str(self.key) + ":" + str(self.val) + ")"
```

key	
val	
parent	
left	right

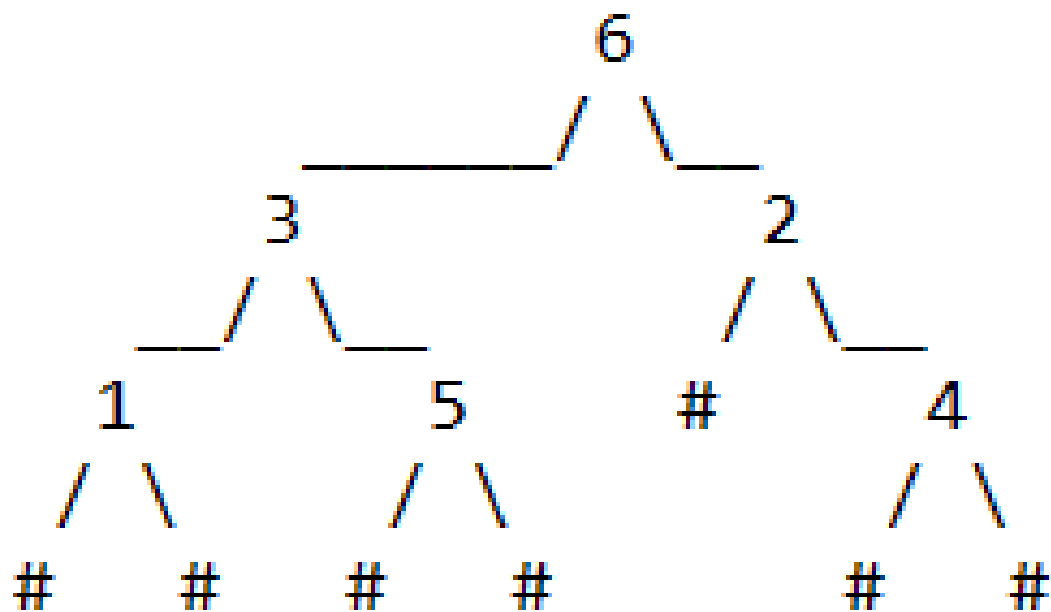
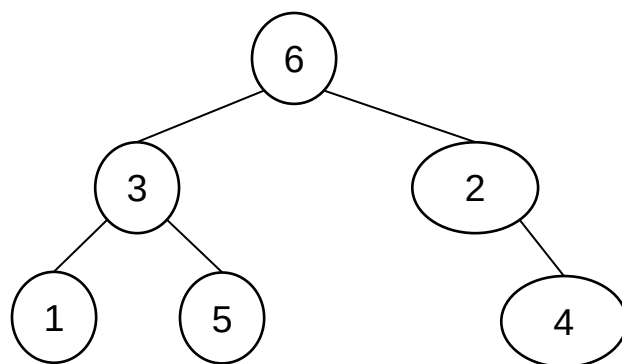
Binary tree class



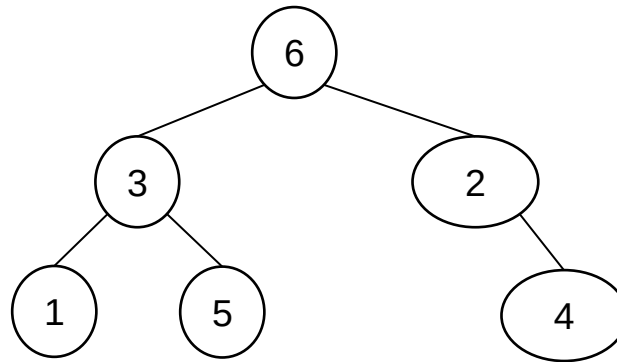
BinaryTree: `__repr__`

- Representation of a tree can be done **recursively**
- Based on implementation by a former TAU student, Amitai Cohen (**No need to understand implementation details**)

```
def __repr__(self):  
    #no need to understand the implementation of this one  
    out = ""  
    #need printree.py file or make sure to run it in the NB  
    for row in printree(self.root):  
        out = out + row + "\n"  
    return out
```



Binary tree traversal



In order

Left subtree
Root
Right subtree

1,3,5,6,2,4

Pre order

Root
Left subtree
Right subtree

6,3,1,5,2,4

Post order

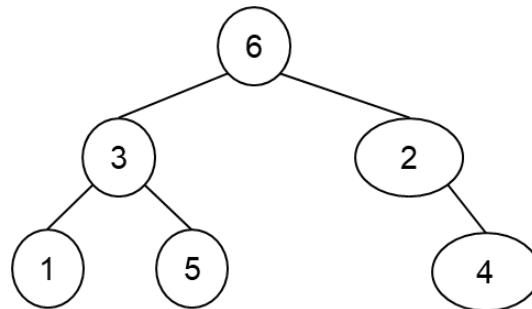
Left subtree
Right subtree
Root

1,5,3,4,2,6

Implementation with generators

```
def in_order_tree_gen_wrapper(tree):  
    for node in in_order_tree_gen(tree.root):  
        yield node  
  
def in_order_tree_gen(node):  
    if not node is None:  
        for node_left in in_order_tree_gen(node.left):  
            yield node_left  
        yield node  
        for node_right in in_order_tree_gen(node.right):  
            yield node_right  
  
for node in in_order_tree_gen_wrapper(bin_tree):  
    print(node)
```

1:a
3:c
5:e
6:f
2:b
4:d

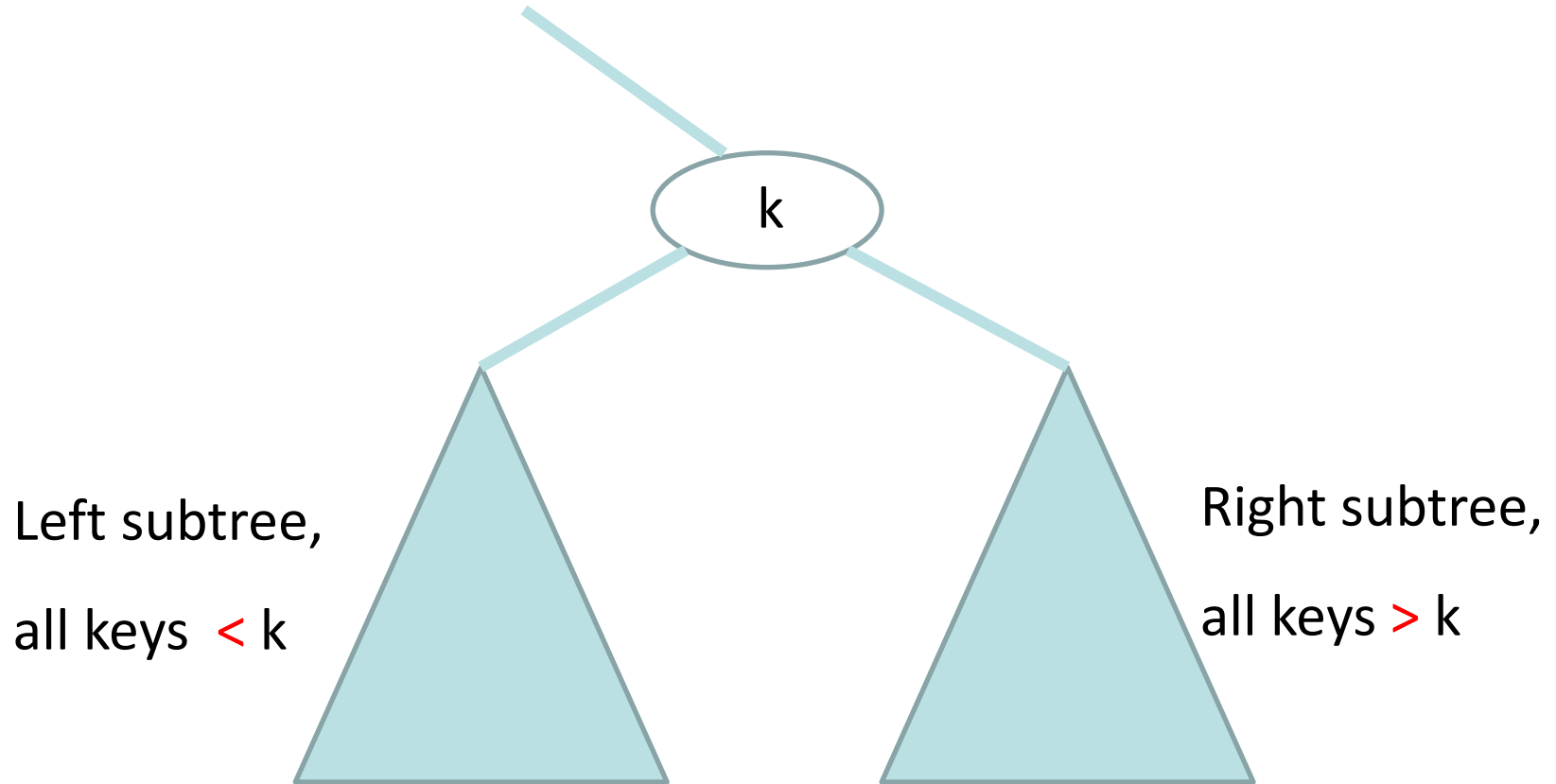


Binary Search Trees (BST)

- Binary **search** trees are data structures used to represent collections of data items
- They support operations like **insert**, **search**, **delete**
- Each node contains a single **data record**. We will assume each record consists of a **key** and **value**
- A node will also include **pointers** to its left and right subtrees, as well as a pointer to its parent
- The **keys** in the binary search tree are organized so that for every node all the keys in the **left/right** subtrees are **smaller/larger** than the key in the current node (assuming unique keys)

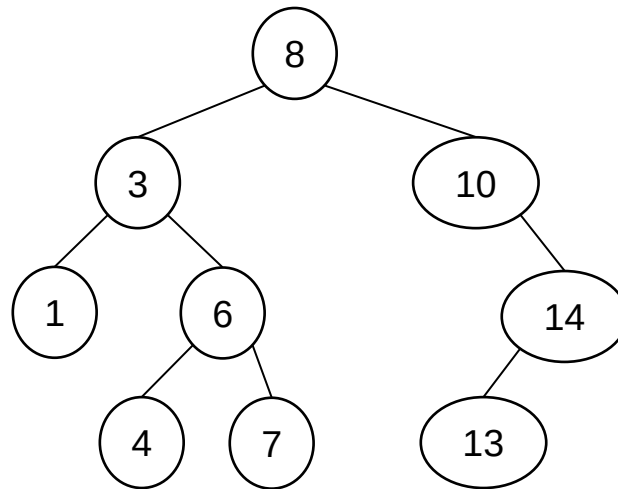
Binary search property

For each node, all the keys in the **left/right** subtrees are **smaller/larger** than the key in the current node, respectively



BST example

Shown are the **keys**, each node also has a **value** (not shown)



BST class

```
class BinarySearchTree():  
    def __init__(self):  
        self.root = None
```

BST: lookup

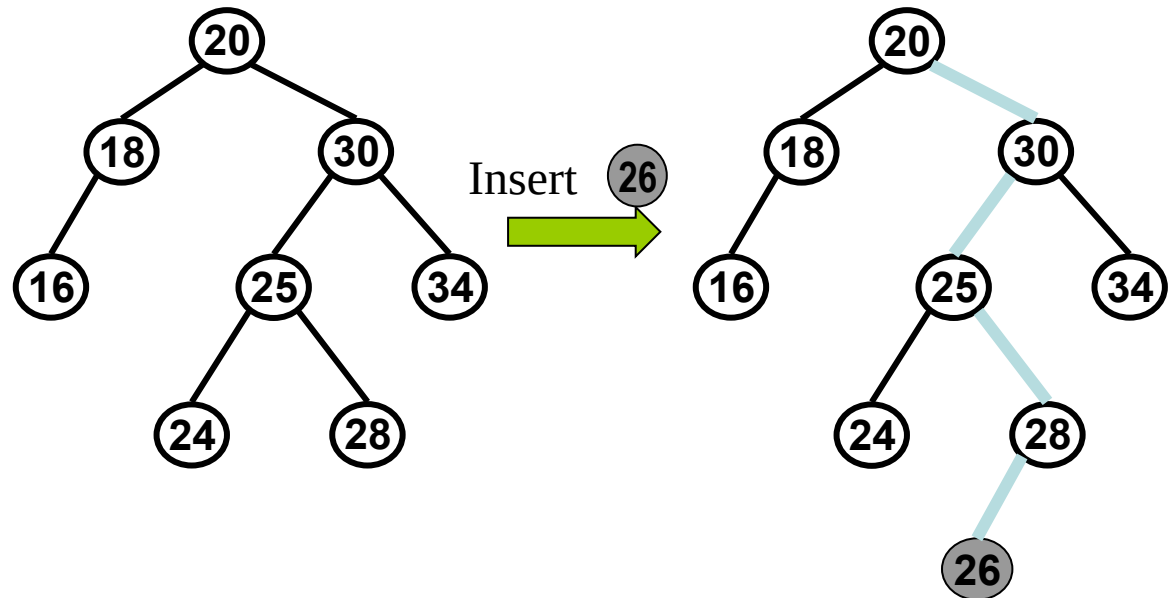
A **wrapper** function that calls a **recursive** function

```
def lookup(self, key):  
    ''' return node with key, uses recursion '''  
  
    def lookup_rec(node, key):  
        if node == None:  
            return None  
        elif key == node.key:  
            return node  
        elif key < node.key:  
            return lookup_rec(node.left, key)  
        else:  
            return lookup_rec(node.right, key)  
  
    return lookup_rec(self.root, key)
```

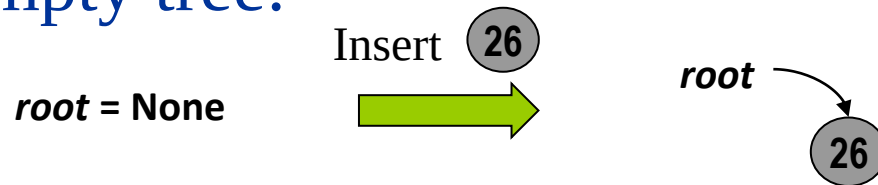
BST: insert

- Look for the appropriate location for insertion
- Hang the new node as a leaf

Example:



Inserting to an empty tree:



BST: insert

- Insert: **wrapper** that calls a **recursive** function
- The recursive function returns None, and modifies the tree with a new leaf
- A key that already exists is replaced (maintaining key uniqueness quality)

BST: insert

```
def insert(self, key, val):
    ''' insert node with key,val into tree, uses recursion '''
    def insert_rec(node, key, val):
        if key == node.key:
            node.val = val      # update the val for this key
        elif key < node.key:
            if node.left == None:
                node.left = TreeNode(key, val)
                node.left.parent = node
            else:
                insert_rec(node.left, key, val)
        else: #key > node.key:
            if node.right == None:
                node.right = TreeNode(key, val)
                node.right.parent = node
            else:
                insert_rec(node.right, key, val)
        return

    if self.root == None: #empty tree
        self.root = TreeNode(key, val)
    else:
        insert_rec(self.root, key, val)
```

Time complexity of lookup / insert

- Lookup follows a path from the root to the node where the element is found, or to a leaf
- Time complexity: the length of the path from the root to the element we are looking for
- **Best case:** when the element looked is in the root $O(1)$
- **Worst case:** when traversing to the farthest leaf in a totally unbalanced tree $O(n)$
- **Balanced trees** (data structures course): $n = 2^{h+1} - 1$, $O(\log n)$
- **Realistically:** Trees tend to be quite balanced if built randomly. A typical tree is of depth $O(\log n)$

Binary search tree:

insert and lookup time complexity

	best case	worst case for any tree	worst case for balanced trees
insert	$O(1)$	$O(n)$	$O(\log n)$
lookup	$O(1)$	$O(n)$	$O(\log n)$

BST: minimum

- Go all the way to the left (maintain two pointers)

```
def minimum(self):  
    ''' return node with minimal key '''  
    if self.root == None:  
        return None  
    node = self.root  
    left = node.left  
    while left != None:  
        node = left  
        left = node.left  
    return node
```

- complexity?
- Alternative: recursion

Time complexity of minimum

- Time complexity: the length of the path from the root to the **leftmost node**
- Best case: when the left subtree is empty $O(1)$
- Worst case: when all right subtrees are empty $O(n)$
- The worst case in a balanced tree is $O(\log n)$

BST: height

- A **recursive** function (two recursive calls)
- By convention:
 - an **empty tree** has a **height of -1**
 - A tree with just a root node has height 0

```
def height(self):  
    ''' return height of tree, uses recursion '''  
    def height_rec(node):  
        if node == None:  
            return -1  
        else:  
            return 1 + max(height_rec(node.left), height_rec(node.right))  
    return height_rec(self.root)
```

Time complexity of height

- Every node is visited **exactly once!**
- A visit to a node takes $O(1)$ time
- Time complexity is always $O(n)$ (num of nodes)

- We can also write a recurrence:

$$T(n) = 1 + T(n_{left}) + T(n_{right})$$

$$T(0) = O(1), T(1) = O(1)$$

where n_{left} and n_{right} are the sizes of the left and right trees, respectively ($n = n_{left} + n_{right} + 1$)

- Solution: $T(n) = O(n)$ (no proof)

BST: size

- Similar to the height...

```
def size(self):  
    ''' return number of nodes in tree, uses recursion '''  
    def size_rec(node):  
        if node == None:  
            return 0  
        else:  
            return 1 + size_rec(node.left) + size_rec(node.right)  
  
    return size_rec(self.root)
```

- Time complexity = $O(n)$
- Or maintain a size attribute for $O(1)$

BST: successor

- Input: Node n in a BST
- Output: the next node in the sorted order

Algorithm:

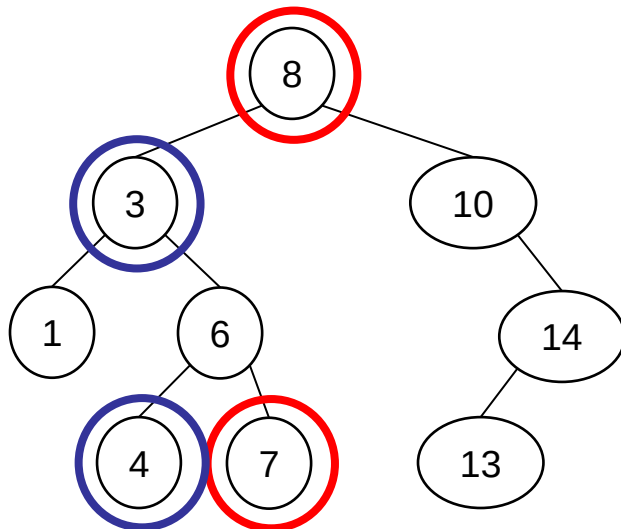
- If $n.right$ go one step right and then left till the end and return that node
- Else return lowest ancestor m of n such that $m.key > n.key$ and return m

In what class should successor be implemented?
BinarySearchTree? TreeNode?

BST: successor examples

Algorithm:

- If $n.right$ go one step right and then left till the end and return that node
- Else go to lowest ancestor m s.t. $m.key > n.key$



Successor implementation in TreeNode

- Input: Node n in a BST
- Output: the next node in the sorted order

```
def successor(self):  
    succ = self  
    if self.right:  
        succ = succ.right  
        while succ.left:  
            succ = succ.left  
    else:  
        while succ and self.key >= succ.key:  
            succ = succ.parent  
  
    return succ
```

- Time complexity = $O(h)$

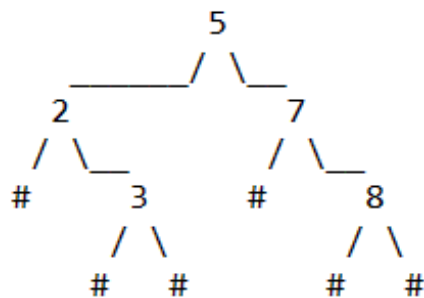
BST traversal with an iterator

- In BST:

```
def __iter__(self):
    self.runner = self.minimum()
    return self

def __next__(self):
    if self.runner is None:
        raise StopIteration
    res = self.runner
    self.runner = self.runner.successor()
    return res
```

```
tree = BinarySearchTree()
tree.insert(5, "a")
tree.insert(2, "b")
tree.insert(3, "c")
tree.insert(7, "d")
tree.insert(8, "e")
print(tree)
```



```
for node in tree:
    print(node)
```

(2:b)
(3:c)
(5:a)
(7:d)
(8:e)

BST: concluding remarks

- Delete a node is harder, omitted here
- Average case time complexity of lookup and insert - Data Structures course
- Balanced tree → efficient store and search data, but tree shape depends on the sequence of inserts
- Variations of balanced binary search trees:
(AVL trees, Red and Black trees)
 - Ensure that the tree remains balanced,
 - Perform balancing operations upon insertion/deletion
(Data Structures course)

Linked data structures

- Two variants of linked data structures: linked lists and trees
- Tradeoffs in terms of complexity
- Next: detecting cycles in linked lists

Set

- Equivalent to sets in mathematics
- Undefined order
- Mutable
- Supports operations: check if an element belongs to a set, add, delete, iterate and standard set operations: union, intersection, difference
- Implementation: similar to dict (using hash tables), key & value are the same...

Examples

```
set('abcdabcd')
```

```
{'a', 'b', 'c', 'd'}
```

```
A = {1, 2, 3}
```

```
B = {3, 2, 3, 1}
```

```
print(A == B)
```

True

```
primes = {2, 3, 5, 7, 11}
for num in primes:
    print(num)
```

2

3

5

7

11

Operations on sets

A B A.union(B)	Returns a set which is the union of sets A and B .
A = B A.update(B)	Adds all elements of array B to the set A .
A & B A.intersection(B)	Returns a set which is the intersection of sets A and B .
A &= B A.intersection_update(B)	Leaves in the set A only items that belong to the set B .
A - B A.difference(B)	Returns the set difference of A and B (the elements included in A , but not included in B).
A -= B A.difference_update(B)	Removes all elements of B from the set A .
A ^ B A.symmetric_difference(B)	Returns the symmetric difference of sets A and B (the elements belonging to either A or B , but not to both sets simultaneously).
A ^= B A.symmetric_difference_update(B)	Writes in A the symmetric difference of sets A and B .
A <= B A.issubset(B)	Returns true if A is a subset of B .
A >= B A.issuperset(B)	Returns true if B is a subset of A .
A < B	Equivalent to A <= B and A != B
A > B	Equivalent to A >= B and A != B

Detecting cycles

```
def detect_cycle1(self)
    s = set()
    p = self.head
    while True:
        if p == None:
            return False
        if p in s:
            print("found ", p, " in:\n",
                  '\n'.join(str(i) for i in s), sep='')
            return True
        s.add(p)
        p = p.next
```

- Add linked list element to s
- Worst case: traverse the whole list to detect a cycle $O(n)$
- Additional memory $O(n)$
- Can we detect cycles using $O(1)$ additional memory?

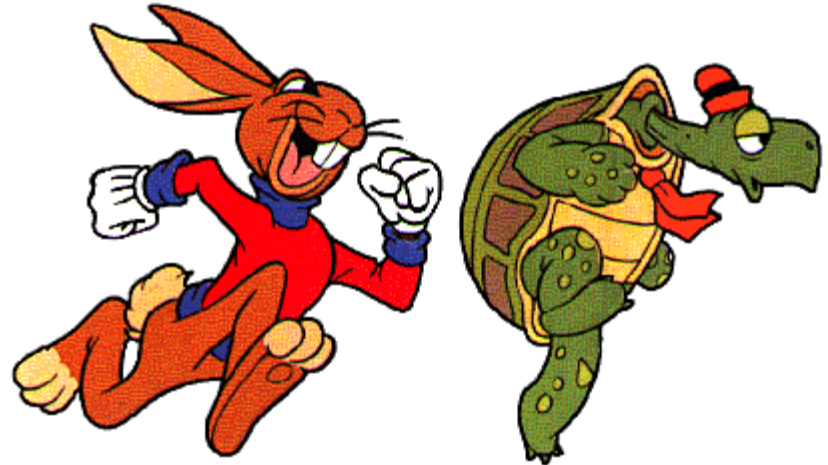
Detecting cycles: Bob Floyd's Tortoise and the Hare Algorithm (1967)

The hare moves twice as quickly as the tortoise

Eventually they will both be inside the cycle. The distance between them will then **decrease by 1** at each additional step

When this distance becomes 0, they are on the same point on the cycle.

See demo on board..



Detecting cycles:

The Tortoise and the Hare Algorithm

```
def detect_cycle2(self):  
    """ 'fast' moves twice as quickly as 'slow'  
    Eventually they will both reach the cycle and the distance  
    between them will decrease by 1 every time until they meet """  
    slow = fast = self.head  
    while True:  
        if slow == None or fast == None:  
            return False  
        elif fast.next == None:  
            return False  
        slow = slow.next  
        fast = fast.next.next  
        print("lst= ", id(self), "slow= ", id(slow), "fast= ", id(fast))  
        if slow is fast:  
            return True
```

Testing the cycle algorithms

```
lst = LinkedList("abcde")  
print(lst)
```

```
nodes: [a, nxt:2742085761032]  
[b, nxt:2742085760520]  
[c, nxt:2742085759496]  
[d, nxt:2742085760904]  
[e, nxt:140726947888352]
```

```
lst.detect_cycle1()
```

```
False
```

```
lst.create_cycle(2, 4)  
# lst - infinite loop..
```

Testing the cycle algorithms

```
lst.detect_cycle1()
```

```
found [c, nxt:2742084812232] in:
```

```
[a, nxt:2742084810504]
```

```
[b, nxt:2742084812680]
```

```
[c, nxt:2742084812232]
```

```
[d, nxt:2742084810376]
```

```
[e, nxt:2742084812680]
```

```
True
```

```
lst.detect_cycle2()
```

```
lst= 2742085788296 slow= 2742084810504 fast= 2742084812680
```

```
lst= 2742085788296 slow= 2742084812680 fast= 2742084810376
```

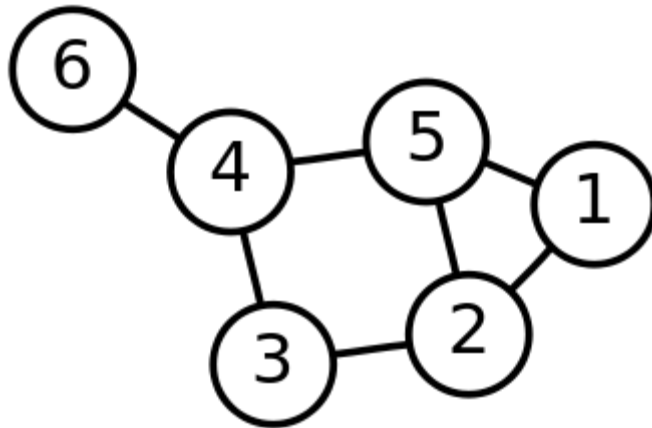
```
lst= 2742085788296 slow= 2742084812232 fast= 2742084812232
```

Linked data structures

- **Linked lists** are the simplest form of linked data structures
- **Doubly-linked** lists: nodes include a pointer to the preceding and to the next item
- **Binary trees**: node points to its left and right child
- **Graphs**: node can point to multiple other nodes, can have cycles (coming next)

Graphs

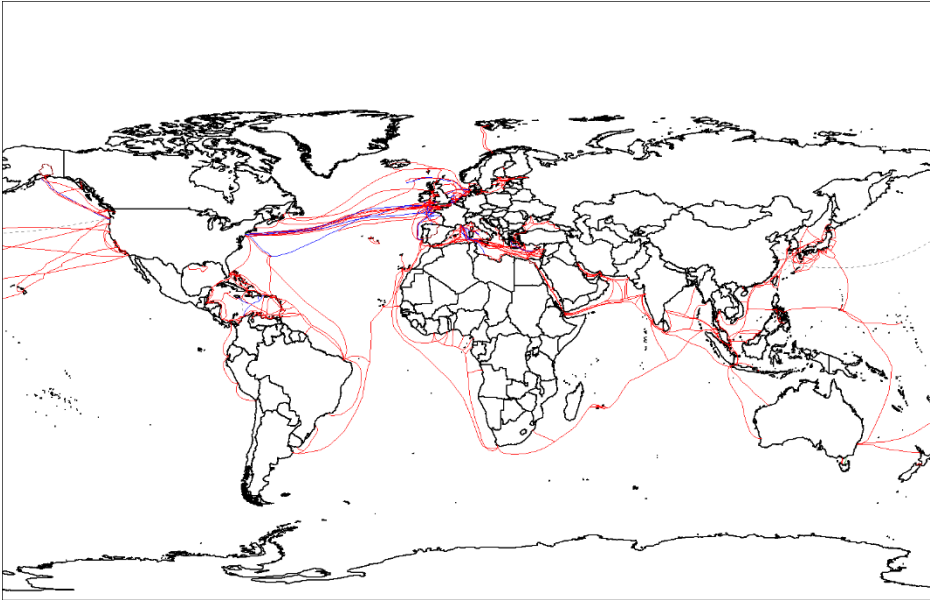
- A graph is a structure with **Nodes** (or vertices) and **Edges**. An edge connects two nodes
- In **directed graphs**, edges have a direction
- In undirected graphs, the edges have no direction



Example of undirected graph

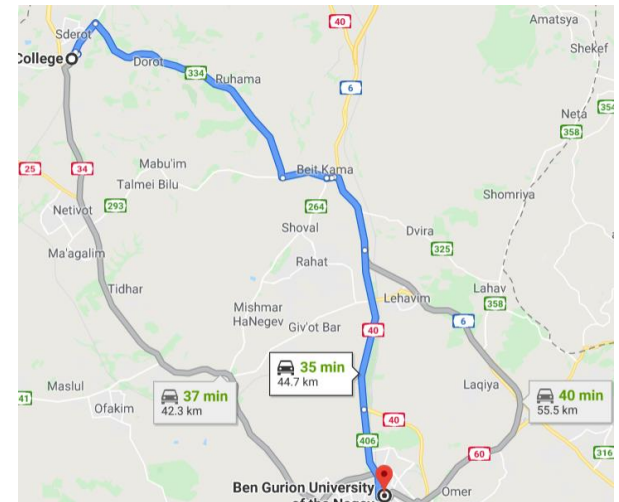
Source: wikipedia

Graphs



The internet
Source: Wikipedia

Social networks
Source: Facebook website



Maps
Source: Google Maps